Tabla de contenido

5 Consultas sobre XML con XPath 2
5.1.Introducción 2
5.2 Rutas y selectores básicos 3
Tipos de rutas 3
Selectores y operadores de navegación 4
Navegación avanzada 6
Funciones básicas aplicadas a rutas 7
5.3. Predicados y filtros 8
Estructura general de un predicado 8
Tipos de predicados 8
Uso de funciones en los filtros 10
Predicados encadenados 10
Evaluaciones booleanas 11
Ejemplos 11
Buenas prácticas 14
5.4 Funciones XPath más utilizadas 14
Funciones numéricas 15
Funciones de cadena de texto 16
Funciones de posición y contexto 16
Funciones booleanas y de comparación 17
Funciones combinadas 18
Recomendaciones de uso 19
5.5 Uso de XPath desde Java 19
Clases principales 19
Flujo general de trabajo 20
Consultas más habituales 21
Evaluación de expresiones XPath en Java 21
Excepciones comunes 24
Resumen 24

5 Consultas sobre XML con XPath

5.1.Introducción

XPath es un lenguaje **de consulta** para documentos **XML**. Permite localizar nodos (elementos, atributos, textos, etc.) usando **rutas** y **filtros**.
No modifica: **solo selecciona**.

- Se usa dentro de: XSLT, XQuery, validaciones, y desde Java (API javax.xml.xpath).
- Piensa en rutas estilo sistema de archivos, con filtros entre corchetes.

En un documento XML, cada parte (elemento, atributo, texto, comentario...) se considera un **nodo** dentro de un árbol.
XPath puede **recorrer este árbol** y devolver diferentes tipos de resultados.

Ejemplo fichero XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Catálogo de productos de ejemplo -->
<catalogo>
    <producto id="P01" categoria="periféricos">
        <nombre>Teclado mecánico</nombre>
        <precio moneda="EUR">49.99</precio>
        <descripcion>Teclado con retroiluminación LED</descripcion>
    </producto>

    <producto id="P02" categoria="periféricos">
        <nombre>Ratón inalámbrico</nombre>
        <precio moneda="EUR">29.50</precio>
        <descripcion>Ratón con sensor óptico de alta precisión</descripcion>
    </producto>

    <producto id="P03" categoria="pantallas">
        <nombre>Monitor 24 pulgadas</nombre>
        <precio moneda="EUR">179.00</precio>
        <descripcion>Resolución Full HD con panel IPS</descripcion>
    </producto>
</catalogo>
```

Un documento XML está compuesto por:

- **Elementos:** <producto>, <nombre>, <precio>
- **Atributos:** id="P01", categoria="periféricos", moneda="EUR"
- **Texto:** Teclado mecánico, 49.99
- **Comentarios:** <!-- Catálogo de productos... -->

- **Nodo raíz:** <catalogo>
- **Instrucción de procesamiento:** <?xml version="1.0"?>

XPath puede devolver:

- **NODE** → un nodo individual
- **NODESET** → conjunto de nodos (lista)
- **STRING** → texto o valor textual
- **NUMBER** → número
- **BOOLEAN** → verdadero/falso

5.2 Rutas y selectores básicos

Las expresiones **XPath** utilizan rutas para moverse por el árbol de nodos del documento XML.
El funcionamiento es muy similar al de las rutas de un sistema de archivos (carpetas y subcarpetas).

Tipos de rutas

Tipo de ruta	Sintaxis	Ejemplo	Descripción
Absoluta	/	/catalogo/producto/nombre	Comienza siempre desde el nodo raíz del documento XML.
Relativa	*(sin / al inicio)*	producto/nombre	Parte del nodo actual (no desde la raíz).
Descendiente	//	//nombre	Selecciona todos los nodos <nombre> sin importar su nivel.
Con atributo	@	//producto/@id	Devuelve los valores de un atributo.

Tipo de ruta	Sintaxis	Ejemplo	Descripción
Mixta	/catalogo//precio	/catalogo//precio	Combina rutas absolutas y descendientes.
Comodín	* o @*	//producto/*	Selecciona todos los elementos o atributos del nodo.

Ejemplos prácticos sobre el XML del catálogo

Consulta XPath	Resultado	Explicación
/catalogo/producto/nombre	Todos los nombres de productos	Ruta **absoluta** desde la raíz <catalogo>.
//nombre	Todos los <nombre> del documento	Ruta **descendiente**, ignora niveles intermedios.
//producto/@categoria	“periféricos”, “pantallas”...	Selecciona el valor del atributo categoria en cada producto.
/catalogo/producto[1]/nombre	“Teclado mecánico”	Selecciona el **primer producto** del catálogo.
(//producto)[last()]/nombre	“Monitor 24 pulgadas”	Devuelve el **último** producto usando last().

Selectores y operadores de navegación

Selector	Significado	Ejemplo	Resultado
/	Hijo directo	/catalogo/producto	Selecciona los elementos <producto> que están **directamente** bajo <catalogo>.
//	Descendiente (en cualquier nivel)	//precio	Selecciona todos los <precio> en el documento.

Selector	Significado	Ejemplo	Resultado
.	Nodo actual	.//nombre	Busca nodos <nombre> desde el nodo actual.
..	Nodo padre	../@id	Devuelve el atributo id del elemento padre.
@	Atributo	//producto/@categoria	Selecciona los valores del atributo categoria.
*	Cualquier elemento	//producto/*	Devuelve todos los hijos de <producto>.

Ejemplos sobre catalogo

Consulta XPath	Resultado	Explicación
/catalogo/producto/nombre	“Teclado mecánico”, “Ratón inalámbrico”, “Monitor 24 pulgadas”	Ruta absoluta desde la raíz.
//nombre	Todos los <nombre> del documento	Ruta descendiente, ignora el nivel jerárquico.
//producto/@id	“P01”, “P02”, “P03”	Devuelve los valores de los atributos id.
(//producto)[last()]/nombre/text()	“Monitor 24 pulgadas”	Devuelve el último producto con la función last().
/catalogo/producto[1]/precio	<precio moneda="EUR">49.99</precio>	Devuelve el nodo completo <precio> del

Consulta XPath	Resultado	Explicación
		primer producto.
//producto/*	Todos los hijos (nombre, precio, descripción)	Usa el comodín *.

Puedes combinar rutas y selectores para recorrer el documento con precisión:

- /catalogo/producto[2]/nombre → nombre del segundo producto.
- //producto[precio > 100]/nombre → nombre de los productos con precio superior a 100 €.
- /catalogo//descripcion → todas las descripciones dentro del catálogo, sin importar el nivel.

Navegación avanzada

Los **ejes** son palabras clave que indican la relación entre el nodo actual y los nodos que se quieren seleccionar.

Eje	Descripción	Ejemplo	Qué devuelve
child::	Hijos directos (por defecto)	child::producto/nombre	Igual que producto/nombre.
parent::	Nodo padre	parent::catalogo	Devuelve el nodo padre del actual.
ancestor::	Antepasados	ancestor::catalogo	Todos los nodos superiores hasta la raíz.
descendant::	Descendientes	descendant::nombre	Todos los nodos <nombre> bajo el actual.
following-sibling::	Hermanos posteriores	//producto[1]/following-sibling::producto	Los productos que van después del primero.

Eje	Descripción	Ejemplo	Qué devuelve
preceding-sibling::	Hermanos anteriores	//producto[3]/preceding-sibling::producto	Los productos que van antes del tercero.
attribute::	Atributos (equivale a @)	attribute::id	Devuelve el atributo id.

Funciones básicas aplicadas a rutas

Las funciones en XPath permiten contar, filtrar, convertir o comparar nodos.

Función	Ejemplo	Descripción
count()	count(//producto)	Cuenta el número total de productos.
sum()	sum(//precio)	Suma los valores numéricos de todos los precios.
position()	//producto[position()=2]/nombre	Devuelve el nombre del segundo producto.
last()	(//producto)[last()]/nombre	Devuelve el nombre del último producto.
name()	name(//producto[1])	Devuelve el nombre del nodo: producto.
starts-with()	//nombre[starts-with(.,'M')]	Devuelve los nombres que empiezan por M.
contains()	//descripcion[contains(.,'LED')]	Selecciona las descripciones que contienen “LED”.

Ejemplo:

Expresión	Resultado esperado
//producto/nombre/text()	Lista con los nombres de los productos

Expresión	Resultado esperado
//producto[precio > 100]/nombre	Nombre de los productos caros
count(//producto)	3
sum(//precio)	258.49
(//producto)[last()]/@id	“P03”
//producto[1]/../@*	Todos los atributos del nodo padre (catalogo)

5.3. Predicados y filtros

Los **predicados** en XPath son **condiciones** que se colocan entre corchetes [ ] para **filtrar nodos** según su posición o su contenido.
Permiten seleccionar solo aquellos elementos que cumplan una o varias condiciones.

Estructura general de un predicado

```
/ruta/nodo[condición]
```

- Se evalúa la condición para cada nodo encontrado.
- Solo se devuelven los nodos que cumplan la condición.
- Pueden combinarse varios predicados consecutivos.

Ejemplo: Selecciona los elementos <producto> cuyo precio sea mayor que 50.

```
//producto[precio > 50]
```

Tipos de predicados

1. Por posición

Permite acceder a nodos según su orden en el documento.

Expresión	Resultado
/catalogo/producto[1]	Primer producto del catálogo.
(//producto)[last()]	Último producto.

Expresión	Resultado
//producto[position() <= 2]	Los dos primeros productos.

position() devuelve el número de posición del nodo dentro del conjunto actual

2. Por valor de un hijo

Filtra nodos en función del contenido de un elemento interno.

Expresión	Resultado
//producto[precio > 100]	Productos con precio superior a 100 €.
//producto[nombre='Ratón inalámbrico']	Producto cuyo nombre sea exactamente “Ratón inalámbrico”.
//producto[descripcion='Resolución Full HD con panel IPS']	Producto con esa descripción exacta.

3. Por valor de un atributo

Los atributos se filtran con @nombreAtributo.

Expresión	Resultado
//producto[@categoria='periféricos']	Productos cuya categoría sea “periféricos”.
//producto[@id='P02']/nombre	Nombre del producto con id “P02”.
//producto[@id!='P01']	Todos los productos excepto el de id P01.

4. Predicados combinados

Se pueden combinar varias condiciones con operadores lógicos and, or y not().

Expresión	Resultado
//producto[@categoria='periféricos' and precio < 50]	Productos de categoría “periféricos” con precio menor que 50 €.

Expresión	Resultado
//producto[@categoria='pantallas' or precio > 100]	Productos que sean pantallas o cuyo precio supere 100 €.
//producto[not(@categoria='pantallas')]	Todos los productos que **no** sean pantallas.

Uso de funciones en los filtros

Las funciones permiten aplicar condiciones más complejas sobre los valores de los nodos o atributos.

Función	Ejemplo	Resultado
contains(cadena1, cadena2)	//producto[contains(nombre, 'Ratón')]	Devuelve los productos cuyo nombre contiene la palabra “Ratón”.
starts-with(cadena1, cadena2)	//producto[starts-with(nombre, 'M')]	Productos cuyo nombre empieza por “M”.
ends-with(cadena1, cadena2) *(solo XPath 2.0)*	//producto[ends-with(nombre, 'co')]	Productos cuyo nombre termina en “co”.
string-length()	//descripcion[string-length(.) > 30]	Descripciones con más de 30 caracteres.
normalize-space()	//producto[normalize-space(nombre)='Teclado mecánico']	Elimina espacios extra antes de comparar.

Predicados encadenados

Los predicados pueden aplicarse de forma **encadenada** para refinar la búsqueda paso a paso.

Ejemplo: Selecciona los productos **periféricos**, con **precio menor que 50** y cuyo **nombre contenga “Ratón”**.

```
//producto[@categoria='periféricos'][precio < 50][contains(nombre, 'Ratón')]
```

Evaluaciones booleanas

Las condiciones dentro de los predicados se pueden usar para generar **valores booleanos** (true o false).

Ejemplo	Devuelve
count(//producto[precio > 100]) > 0	true si hay al menos un producto con precio > 100.
count(//producto[@categoria='pantallas']) = 1	true si solo existe una pantalla.

Ejemplos

Consulta XPath	Resultado esperado	Explicación
//producto[@id='P02']/nombre/text()	Ratón inalámbrico	Devuelve el nombre del producto con identificador P02.
//producto[precio > 50]/nombre/text()	Monitor 24 pulgadas	Selecciona los productos cuyo precio es superior a 50 €.
//producto[@categoria='periféricos']/nombre/text()	Teclado mecánico, Ratón inalámbrico	Filtra los productos por el valor del atributo categoria.
(//producto)[1]/nombre/text()	Teclado mecánico	Devuelve el nombre del **primer producto** del documento.

Consulta XPath	Resultado esperado	Explicación
(//producto)[last()]/nombre/text()	Monitor 24 pulgadas	Devuelve el nombre del **último producto** del catálogo.
//producto[position()=2]/precio/text()	29.50	Devuelve el precio del **segundo producto**.
//producto[precio < 50]/@id	P01, P02	Devuelve los identificadores de los productos con precio menor que 50 €.
//producto[precio > 30 and @categoria='periféricos']/nombre/text()	*(ninguno)*	No hay periféricos con precio mayor de 30 €.
//producto[precio < 50 and contains(nombre, 'Ratón')]/nombre/text()	Ratón inalámbrico	Combina condiciones sobre texto y valor numérico.
//producto[starts-with(nombre,'M')]/nombre/text()	Monitor 24 pulgadas	Devuelve los productos cuyo nombre empieza por M.
//producto[contains(descripcion,'LED')]/nombre/text()	Teclado mecánico	Busca productos cuya descripción

Consulta XPath	Resultado esperado	Explicación
		contiene la palabra “LED”.
//producto[not(@categoria='pantallas')]/nombre/text()	Teclado mecánico, Ratón inalámbrico	Excluye los productos de la categoría “pantallas”.
//producto[@categoria='periféricos'][precio < 50]/nombre/text()	Teclado mecánico, Ratón inalámbrico	Filtro doble: categoría y valor inferior a 50 €.
//producto[@id!='P01']/nombre/text()	Ratón inalámbrico, Monitor 24 pulgadas	Devuelve todos los productos excepto el de id P01.
//producto[precio>20 and precio<100]/nombre/text()	Teclado mecánico, Ratón inalámbrico	Filtro por rango de precios.
//producto[precio > 50 or contains(descripcion,'Full HD')]/nombre/text()	Monitor 24 pulgadas	Condición compuesta con **or**.
//producto[string-length(descripcion) > 30]/nombre/text()	Ratón inalámbrico, Monitor 24 pulgadas	Devuelve productos con descripciones largas.
//producto[normalize-space(nombre)='Teclado mecánico']/@id	P01	Normaliza espacios antes de comparar texto.

Consulta XPath	Resultado esperado	Explicación
//producto[precio = max(//precio)]/nombre/text() *(solo XPath 2.0)*	Monitor 24 pulgadas	Selecciona el producto más caro.
count(//producto[@categoria='periféricos'])	2	Devuelve el número de productos periféricos.
sum(//precio)	258.49	Calcula la suma de los precios de todos los productos.
//producto[precio = 29.50]/../@*	*(sin resultado visible en texto)*	Devuelve todos los atributos del nodo padre (catalogo).

Buenas prácticas

- Usa **comillas simples o dobles** según convenga ([@id='P01'] o [@id="P01"]).
- Las comparaciones son **sensibles a mayúsculas y minúsculas**.
- Si hay posibles espacios, usa normalize-space() antes de comparar.
- Evita combinaciones ambiguas: (//producto)[1] y //producto[1] **no son equivalentes**.
    - (//producto)[1] → el primer producto de todo el documento.
    - //producto[1] → el primer producto **de cada grupo de hermanos** bajo su padre.

5.4 Funciones XPath más utilizadas

Además de los predicados y rutas, XPath incluye una serie de **funciones integradas** que permiten realizar operaciones sobre los nodos y valores del

documento XML.
Estas funciones facilitan la búsqueda, el filtrado y la manipulación de los resultados sin necesidad de modificar el XML.

Clasificación general de funciones

Tipo de función	Ejemplos de funciones más comunes
Numéricas	count(), sum(), floor(), ceiling(), round()
De cadena de texto	string(), concat(), contains(), starts-with(), string-length(), normalize-space()
De posición y contexto	position(), last(), name()
Booleanas (verdadero/falso)	not(), boolean(), operadores and, or
De nodo	local-name(), namespace-uri(), name()

Funciones numéricas

Función	Ejemplo	Resultado	Explicación
count()	count(//producto)	3	Devuelve el número total de productos.
sum()	sum(//precio)	258.49	Suma todos los valores numéricos de los nodos <precio>.
floor()	floor(//precio[1])	49	Redondea hacia abajo el valor del primer precio.
ceiling()	ceiling(//precio[2])	30	Redondea hacia arriba el precio del segundo producto (29.50 → 30).
round()	round(sum(//precio) div count(//precio))	86	Calcula el promedio de precios y redondea el resultado.

Funciones de cadena de texto

Función	Ejemplo	Resultado	Explicación
string()	string(//producto[@id='P01']/nombre)	Teclado mecánico	Convierte el nodo seleccionado a texto.
concat()	concat(//producto[1]/nombre, ' - ', //producto[1]/precio)	Teclado mecánico - 49.99	Une varios textos o valores.
contains()	//producto[contains(descripcion, 'LED')]/nombre	Teclado mecánico	Comprueba si una cadena contiene otra.
starts-with()	//producto[starts-with(nombre,'R')]/nombre	Ratón inalámbrico	Devuelve los productos cuyo nombre empieza por “R”.
string-length()	string-length(//descripcion[1])	35	Devuelve el número de caracteres del texto.
normalize-space()	//producto[normalize-space(nombre)='Teclado mecánico']/@id	P01	Elimina espacios extra al inicio y final del texto antes de comparar.

Funciones de posición y contexto

Estas funciones devuelven información sobre **la posición de un nodo dentro de su conjunto** o sobre el nombre del nodo actual.

Función	Ejemplo	Resultado	Explicación
position()	//producto[position()=2]/nombre/text()	Ratón inalámbrico	Devuelve el nodo que ocupa la posición 2.
last()	(//producto)[last()]/@id	P03	Devuelve el último producto del documento.
name()	name(//producto[1])	producto	Devuelve el nombre del nodo (etiqueta).
local-name()	local-name(//precio[1])	precio	Similar a name(), sin el prefijo de espacio de nombres.

Funciones booleanas y de comparación

Permiten evaluar condiciones que devuelven **verdadero (true)** o **falso (false)**.
Son útiles dentro de predicados o para comprobar si existen coincidencias.

Función / operador	Ejemplo	Resultado	Explicación
boolean()	boolean(//producto[@id='P01'])	true	Devuelve true si el nodo existe.
not()	not(//producto[@id='P05'])	true	Devuelve true si **no** existe el nodo.
and	//producto[@categoria='periféricos' and precio < 50]	Teclado mecánico, Ratón inalámbrico	Devuelve productos que cumplan ambas condiciones.
or	//producto[@categoria='pantallas' or precio > 100]	Monitor 24 pulgadas	Cumple al menos una de las condiciones.

Función / operador	Ejemplo	Resultado	Explicación
=	//producto[precio = 49.99]/@id	P01	Compara valores exactos.
!=	//producto[@id!='P03']/nombre	Teclado mecánico, Ratón inalámbrico	Diferente de...

Funciones combinadas

Se pueden combinar funciones para crear consultas más elaboradas:

Ejemplo	Resultado	Descripción
round(sum(//precio) div count(//precio))	86	Calcula el precio medio redondeado.
//producto[string-length(nombre) > 10]/nombre	Ratón inalámbrico, Monitor 24 pulgadas	Devuelve nombres con más de 10 caracteres.
//producto[starts-with(nombre,'M') and precio > 100]/nombre	Monitor 24 pulgadas	Combina texto y número en la misma condición.
count(//producto[contains(descripcion,'sensor')])	1	Cuenta los productos cuya descripción incluye “sensor”.

Recomendaciones de uso

- Las funciones pueden usarse **en predicados [ ]** o **en el cuerpo de la expresión**.
  Ejemplo: sum(//precio) devuelve un número; //producto[precio > sum(//precio) div 3] usa la función dentro del filtro.
- XPath 1.0 solo admite **funciones básicas**. XPath 2.0 y superiores añaden funciones como upper-case(), lower-case(), ends-with(), avg(), max(), min(), etc.
- En entornos Java, las funciones devuelven distintos tipos (String, Number, Boolean, NodeList) según cómo se evalúe la expresión con XPathConstants.

5.5 Uso de XPath desde Java

XPath se utiliza en Java para **consultar información dentro de un documento XML** sin tener que recorrerlo manualmente nodo a nodo.
Para ello, Java proporciona la **API estándar javax.xml.xpath**, que permite evaluar expresiones XPath sobre un árbol DOM y obtener resultados de distintos tipos.

Objetivo

Aplicar consultas XPath dentro de programas Java para:

- Localizar nodos y valores específicos en un documento XML.
- Contar, filtrar y extraer datos sin modificar el archivo original.
- Automatizar tareas de análisis, búsqueda o validación de información.

Clases principales

Clase / Interfaz	Paquete	Función principal
DocumentBuilderFactory	javax.xml.parsers	Crea un analizador DOM que carga el XML en memoria.
Document	org.w3c.dom	Representa el árbol completo del documento XML.
XPathFactory	javax.xml.xpath	Crea objetos XPath que evaluarán las expresiones.

Clase / Interfaz	Paquete	Función principal
XPath	javax.xml.xpath	Objeto principal para compilar y ejecutar expresiones XPath.
XPathExpression	javax.xml.xpath	Permite compilar una expresión XPath y reutilizarla varias veces.
XPathConstants	javax.xml.xpath	Define los tipos de resultados posibles (STRING, NUMBER, BOOLEAN, NODESET).

Flujo general de trabajo

1. **Cargar el documento XML**
   Se utiliza DocumentBuilderFactory para leer el archivo XML y generar un objeto Document.
   Este documento representa todo el árbol de nodos sobre el que se aplicarán las consultas.
2. **Crear el objeto XPath**
   Mediante XPathFactory.newInstance().newXPath() se crea el motor que evaluará las expresiones XPath.
3. **Compilar o evaluar expresiones**
   Las consultas se formulan como cadenas de texto, por ejemplo:

```
//producto[precio>50]/nombre/text()
```

4. **Obtener los resultados**
   Cada expresión puede devolver distintos tipos de datos, según se indique al evaluarla con los **XPathConstants**:

Tipo de resultado	Constante	Descripción
Cadena (String)	XPathConstants.STRING	Devuelve el texto o valor de un único nodo.
Número (Number)	XPathConstants.NUMBER	Devuelve un número (por ejemplo, con count() o sum()).
Booleano (Boolean)	XPathConstants.BOOLEAN	Devuelve verdadero o falso según la condición.

Tipo de resultado	Constante	Descripción
Conjunto de nodos (NodeList)	XPathConstants.NODESET	Devuelve una lista de nodos coincidentes.

Consultas más habituales

Tipo de consulta	Ejemplo XPath	Tipo de resultado
Seleccionar nodos	//producto/nombre/text()	NODESET
Contar nodos	count(//producto)	NUMBER
Sumar valores	sum(//precio)	NUMBER
Buscar texto	//producto[contains(descripcion,'LED')]	NODESET
Comprobar existencia	count(//producto[precio>100])>0	BOOLEAN
Extraer un valor concreto	(//producto)[1]/@id	STRING

Evaluación de expresiones XPath en Java

Una vez que el documento XML ha sido cargado en memoria como un objeto Document, y se ha creado un evaluador XPath, el siguiente paso consiste en **evaluar las expresiones XPath** para obtener información concreta del XML.

La evaluación de una expresión XPath en Java se realiza mediante el método:

```
xpath.evaluate(String expression, Object item, QName returnType)
```

Parámetros:

Parámetro	Tipo	Descripción
expression	String	Es la expresión XPath que se desea ejecutar, escrita como texto. Ejemplo: "//producto[precio>50]/nombre/text()".
item	Object	Nodo o documento sobre el que se ejecuta la consulta. Normalmente es un objeto Document, aunque también puede ser un nodo concreto (Element) si se quiere aplicar la expresión sobre una parte del XML.
returnType	QName	Indica el tipo de resultado que se desea obtener. Se define con una constante de la clase XPathConstants (por ejemplo, STRING, NUMBER, NODESET o BOOLEAN).

El método devuelve un **objeto genérico** que debe convertirse (cast) al tipo correspondiente según el tipo de resultado solicitado.

Tipos de resultado disponibles

XPath puede devolver distintos tipos de datos según el tipo de consulta:

Tipo de resultado	Constante (XPathConstants)	Tipo devuelto en Java	Ejemplo de expresión	Resultado esperado
Cadena de texto (String)	STRING	String	(//producto)[1]/nombre/text()	"Teclado mecánico"
Número	NUMBER	Double	count(//producto)	3.0
Booleano	BOOLEAN	Boolean	count(//producto[precio>100]) > 0	true o false
Conjunto de nodos (NodeSet)	NODESET	NodeList	//producto/nombre/text()	Lista de nodos <nombre>

Nota: XPath 1.0 solo devuelve estos tipos básicos. XPath 2.0 añade otros como secuencias, fechas o tipos específicos, pero no están disponibles en la API estándar de Java SE.

EJEMPLOS:

1)S**eleccionar un conjunto de nodos:**

- Expresión:

```
//producto[precio > 50]/nombre/text()
```

- Tipo de resultado: NODESET
- Descripción: devuelve una lista (NodeList) con los textos de todos los nombres de productos cuyo precio sea mayor que 50.

2) **Obtener un valor concreto (texto):**

- Expresión:

```
(//producto)[1]/nombre/text()
```

- Tipo de resultado: STRING
- Descripción: devuelve el texto del primer nombre encontrado (“Teclado mecánico”).

3) **Calcular un valor numérico:**

- Expresión:

```
sum(//precio)
```

- Tipo de resultado: NUMBER
- Descripción: devuelve la suma total de los precios como número decimal (Double).

4)**Comprobar una condición:**

- Expresión:

```
count(//producto[@categoria='pantallas']) > 0
```

- Tipo de resultado: BOOLEAN
- Descripción: devuelve true si existe al menos un producto en la categoría “pantallas”.

Contexto de evaluación

El parámetro item define **desde qué nodo** del XML se ejecutará la consulta:

- Si se pasa el **documento completo** (Document doc), la consulta se evalúa desde la raíz.
- Si se pasa un **nodo específico** (Element producto), la expresión se evalúa desde ese nodo.
  Por ejemplo, xpath.evaluate("precio/text()", producto, STRING) devuelve el precio del producto actual dentro de un bucle.

Esto permite **reutilizar expresiones** sobre distintos nodos, sin tener que recorrer el árbol manualmente.

Compilación y reutilización de expresiones

Cuando una expresión se va a ejecutar repetidamente (por ejemplo, en un bucle), es más eficiente **compilarla previamente** mediante un objeto XPathExpression:

```
XPathExpression expr = xpath.compile("//producto[precio > 
50]/nombre/text()");
NodeList lista = (NodeList) expr.evaluate(doc, XPathConstants.NODESET);
```

Ventajas:

- Evita que Java tenga que analizar y preparar la expresión cada vez.
- Mejora el rendimiento en aplicaciones que procesan grandes volúmenes de datos XML.

Excepciones comunes

Excepción	Causa habitual	Solución recomendada
XPathExpressionException	La expresión XPath contiene un error de sintaxis o referencia a nodos inexistentes.	Revisar la expresión, especialmente comillas, paréntesis o funciones.
ClassCastException	Se intenta convertir el resultado a un tipo incorrecto.	Verificar que el returnType coincide con el tipo de dato esperado.
NullPointerException	Se evalúa una expresión sobre un documento o nodo no inicializado.	Comprobar que el documento se haya cargado correctamente.

Resumen

Acción	Clase Java implicada
Cargar el XML en un árbol DOM	DocumentBuilderFactory → Document
Crear el evaluador XPath	XPathFactory → XPath
Evaluar la expresión	XPath.evaluate()
Especificar tipo de resultado	XPathConstants.STRING, NUMBER, BOOLEAN, NODESET

Acción	Clase Java implicada
Procesar el resultado	Usar el tipo correspondiente (String, Double, Boolean, NodeList)

Conclusión:
La evaluación de expresiones XPath en Java proporciona una forma **eficiente y declarativa** de acceder a la información de un XML.
En lugar de recorrer el árbol DOM con bucles y condiciones, basta con escribir una **consulta XPath** precisa y elegir el tipo de resultado que se necesite.